

Contents

Python API Overview	5
Import Pattern	6
Coverage Status	6
PDF Export	6
phynexis.utils	7
Import	7
Module Overview	7
C++ Namespace	7
pybind Module	7
Console	8
Config	8
Constructor	8
Properties	8
Methods	8
Logging Functions	9
debug(msg)	9
info(msg)	9
plain(msg)	9
warning(msg)	9
error(msg)	9
fatal(msg)	9
Broadcast Variants	9
Control Functions	10
set_level(level)	10
set_verbose(enabled)	10
set_color(enabled)	10
custom(config)	10
Cork	11
Description	11
Methods	11
mesh_intersect(v1, f1, v2, f2, out_v, out_f, out_info)	11
mesh_union(v1, f1, v2, f2, out_v, out_f, out_info)	11
mesh_difference(v1, f1, v2, f2, out_v, out_f, out_info)	11
mesh_xor(v1, f1, v2, f2, out_v, out_f, out_info)	11
Note	11
Unexposed C++ API	11
DataIO	12
Methods	12
import_data(filename, lines_to_skip=0)	12
save_data(data, head, filename)	12
file_exist(filename)	12
Example	12
Unexposed C++ API	13
FlatHashMap	14
Available Instantiations	14
Constructor	14
FlatHashMap()	14
FlatHashMap(bucket_count)	14

Methods	14
empty()	14
size() / __len__()	14
max_size()	14
reserve(count)	14
clear()	14
insert(key, value)	15
erase(key)	15
find(key)	15
contains(key) / key in map	15
count(key)	15
at(key)	15
load_factor()	15
max_load_factor() / max_load_factor(ml)	15
rehash(count)	15
Python Protocols	15
map[key] (getitem)	15
map[key] = value (setitem)	15
key in map (contains)	15
for k, v in map (iteration)	15
len(map)	16
Example	16
Unexposed C++ API	18
Binding Fix Record	18
LevelSetField	19
Constructor	19
LevelSetField()	19
Methods	19
get_corner()	19
get_spacing()	19
get_dimensions()	19
set_corner(corner)	19
set_spacing(sp)	19
set_dimension(dim)	19
get_phi(id) / set_phi(id, value) / phi(i, j, k)	20
init_from_sdf_calculator(sdf)	20
signed_distance(pos)	20
gradient_interpolate(pos)	20
gradient_minus(id) / gradient_plus(id)	20
reinitialization() / reinitialization(iterations, dt)	20
is_initialized()	20
Example	20
Unexposed C++ API	21
math	22
Constants	22
Vector Operations	22
dot(a, b)	22
cross(a, b)	22
norm_l2(v)	22
normalize(v)	22
Quaternion	22
from_rodrigues(angle, axis)	23

to_rodrigues(q) / to_matrix(q) / from_matrix(m)	23
multiply(a, b) / conjugate(q) / normalize(q) / add(a, b)	23
Example	23
Unexposed C++ API	23
MiniMap	24
Constructor	24
Methods	24
exist(key)	24
size()	24
clear()	24
Python Protocols	24
map[key] (getitem)	24
map[key] = value (setitem)	24
key in map (contains)	24
Example	24
Unexposed C++ API	25
OpenMP	26
Functions	26
omp_get_max_threads()	26
omp_set_num_threads(num_threads)	26
parallel_max_threads()	26
parallel_set_num_threads(num_threads)	26
Example	26
Note	26
sampling	27
Distribution Sampling	27
Spatial Sampling	27
Orientation Sampling	27
Spherical Sampling	27
Example	28
Unexposed C++ API	28
SaveOptions / LoadOptions	29
FileFormat	29
SaveOptions	29
Constructor	29
Properties	29
SaveVtkOptions	29
Constructor	29
Properties	29
LoadOptions	30
Constructor	30
Properties	30
Example	30
SDFCalculator	31
Constructor	31
SDFCalculator()	31
Methods	31
init_from_stl(vertices, facets)	31
init_from_stl(stl_model)	31
initialize()	31

signed_distance(pos)	31
surface_projection(pos)	31
closest_facet(pos)	31
signed_distances(points)	32
Example	32
Unexposed C++ API	32
STLModel	33
Constructor	33
STLModel()	33
STLModel(vertices, facets)	33
STLModel(file_path)	33
Properties	33
Methods	33
init_from_stl(file_path) / init_from_off(file_path)	33
save_to(path, file, options)	33
transpose() / rotate(rotation)	33
copy_pose() / copy_pose_dev()	33
remove_unreferenced_vertices() / remove_duplicate_vertices()	33
reorient_facets()	34
decimate(target_num_faces)	34
standardize()	34
set_size(size)	34
make_convex()	34
refine() / smooth_mesh(iterations)	34
merge(other_model)	34
size()	34
get_center() / get_surface_area() / volume() / inertia()	34
is_convex() / is_face_outside(facet_index, point) / enclose(point)	34
bound_aabb()	34
print_info()	34
Static Methods	34
CenterOf(vertices, facets) / SurfaceAreaOf(vertices, facets) / VolumeOf(vertices, facets) / InertiaOf(vertices, facets) / check_convexity_of(vertices, facets)	34
Example	35
Example: Create from vertices and facets	35
Unexposed C++ API	36
StringUtils	37
Functions	37
to_string(value)	37
Unexposed C++ API	37
SurfaceMesh	38
Constructor	38
SurfaceMesh()	38
Properties	38
Example	38
Unexposed C++ API	38
TimeUtils	39
Functions	39
get_time_micros()	39
Unexposed C++ API	39

Vectors and Matrices	40
Vector Types	40
Constructors	40
Properties / Indexing	40
Methods	40
Matrix Types	41
Constructors	41
Indexing	41
Example	41
Known Issues	42
Unexposed C++ API	42
Voronoi	43
TessellationParams	43
Constructor	43
Properties	43
Tessellation	43
compute(vt_seeds, stl_model, params)	43
compute_centroidal(num_seeds, stl_model, params)	43
Diagram	43
Example	44
Unexposed C++ API	44
vtk	45
Data Classes	45
Read Functions	45
Write Functions	45
Format Detection	45
Example	46
Known Issues	46
Unexposed C++ API	46
wrappers	47
CorkWrapper	47
EigenWrapper	47
IGLWrapper	47
CGALWrapper	48
mpi	48
Example	48
Known Issues	48
Unexposed C++ API	49

Python API Overview

Phynexis exposes its C++ core through pybind11 bindings, organized into 9 submodules:

Module	Python Import	C++ Namespace	Description
utils	phynexis.utils	phynexis::utils	Utilities: console, SDF, string helpers
fields	phynexis.fields	phynexis::fields	Field quantities and level sets
parsim	phynexis.parsim	phynexis::parsim	Parallel simulation framework

Module	Python Import	C++ Namespace	Description
netdem	<code>phynexis.netdem</code>	<code>phynexis::netdem</code>	Discrete Element Method (DEM)
netfem	<code>phynexis.netfem</code>	<code>phynexis::netfem</code>	Finite Element Method (FEM)
cfddem	<code>phynexis.cfddem</code>	<code>phynexis::cfddem</code>	CFD-DEM coupling
ml	<code>phynexis.ml</code>	<code>phynexis::ml</code>	Machine learning interfaces
peridigm	<code>phynexis.peridigm</code>	<code>phynexis::peridigm</code>	Peridynamics
workflow	<code>phynexis.workflow</code>	<code>phynexis::workflow</code>	Workflow automation

Import Pattern

```
import phynexis
```

```
# Lazy-loaded submodules
```

```
scene = phynexis.netdem.Scene()
```

```
mesh = phynexis.netfem.TetMesh()
```

Coverage Status

See progress tracker for the current documentation coverage by module.

PDF Export

A compiled PDF of the full Python API reference is available for download:

Download [phynexis-python-api.pdf](#)

phynexis.utils

Utility module providing helper classes and functions for console logging, string manipulation, time measurement, signed distance fields, mesh I/O, sampling, math, and more.

Import

```
import phynexis

# Access submodule directly
phynexis.utils.info("hello")

# Or bind locally
from phynexis import utils
utils.info("hello")
```

Module Overview

Class / Function	Description
Config	Console output configuration
debug / info / warning / error / fatal / plain	Logging functions
set_level / set_verbose / set_color	Console control functions
to_string	Numeric to string conversion
get_time_micros	High-resolution timestamp
MiniMapIdx64	Lightweight index map
SDFCalculator	Signed distance field calculator
LevelSetField	Level set field representation
STLModel	3D triangle mesh from STL/OFF
Voronoi	Voronoi tessellation (Tessellation / Diagram)
Cork	Boolean mesh operations (CorkWrapper)
OpenMP	Thread control bindings
Vec2d/Vec3d/Vec3i/Vec3u/Vec4d/Vec6d	Fixed-size vectors
Mat2d/Mat3d	Fixed-size matrices
FlatHashMap	Flat hash map
Shape	Geometric shapes (Sphere, Cuboid, Ellipsoid, Cylinder, ...)
SurfaceMesh	Surface mesh struct
DataIO	Tabular data I/O
SaveOptions / LoadOptions	File save/load options
math	Math constants, vector ops, quaternion
sampling	Distribution, spatial, orientation, spherical sampling
vtk	VTK file I/O
wrappers	Cork, CGAL, Eigen, igl, TetGen wrappers

C++ Namespace

phynexis::utils

pybind Module

pyutils (lazy-loaded via phynexis.utils)

Console

C++: phynexis::utils::console **Python:** phynexis.utils **Header:** src/utils/console.hpp

Console logging and output control for phynexis. Supports leveled output (debug/info/warning/error/fatal), colored terminal output, timestamps, and MPI rank display.

Config

C++: phynexis::utils::console::Config **Python:** phynexis.utils.Config

Configuration object controlling the appearance and filtering of console output.

Constructor

Config() Creates a config with default settings (min_level=info, timestamps on, colors on).

Properties

Property	Type	Access	Description
min_level	int	read/write	Minimum log level to display
show_timestamp	bool	read/write	Show timestamps in output
show_level	bool	read/write	Show level tag (INFO, WARN, etc.)
show_location	bool	read/write	Show file/line location
color_output	bool	read/write	Enable ANSI color codes
show_rank	bool	read/write	Show MPI rank prefix

Methods

set_min_level(level) Set minimum display level. Levels: 0=debug, 1=info, 2=warning, 3=error, 4=fatal.

Parameters: | Parameter | Type | Description | |---|---|---| | level | int | Minimum level to display |

set_show_timestamp(enabled) Toggle timestamp display.

set_show_level(enabled) Toggle level tag display.

set_show_location(enabled) Toggle source location display.

set_color_output(enabled) Toggle colored output.

Example:

```
import phynexis

cfg = phynexis.utils.Config()
cfg.set_show_timestamp(False)
cfg.set_color_output(False)
phynexis.utils.custom(cfg)

phynexis.utils.info("no timestamp, no color")
```

Output:

[INFO] no timestamp, no color

Logging Functions

Module-level functions for leveled console output.

`debug(msg)`

Print a debug-level message. Only visible when `min_level ≤ 0`.

`info(msg)`

Print an info-level message.

`plain(msg)`

Print a plain message without any level prefix or formatting.

`warning(msg)`

Print a warning-level message.

`error(msg)`

Print an error-level message.

`fatal(msg)`

Print a fatal-level message and terminate.

Parameters: | Parameter | Type | Description | | `msg` | `str` | Message string to print |

Broadcast Variants

Each level has an `_all` variant that broadcasts to all MPI ranks:

- `debug_all(msg)`
- `info_all(msg)`
- `warning_all(msg)`
- `error_all(msg)`
- `fatal_all(msg)`

Example:

```
import phynexis
```

```
phynexis.utils.info("simulation started")
phynexis.utils.debug("debug info: dt=1e-5")
phynexis.utils.warning("mesh quality low")
phynexis.utils.plain("raw output without prefix")
```

Output:

```
[INFO] [12:01:05.232] simulation started
[DEBUG] [12:01:05.233] debug info: dt=1e-5
[WARNING] [12:01:05.233] mesh quality low
raw output without prefix
```

Control Functions

`set_level(level)`

Set the global minimum log level.

`set_verbose(enabled)`

Enable or disable verbose (debug) output. Equivalent to `set_level(0)`.

`set_color(enabled)`

Enable or disable colorized output globally.

`custom(config)`

Apply a custom Config object to the console.

Example:

```
import phynexis

phynexis.utils.set_level(2)    # Only warning and above
phynexis.utils.info("hidden")
phynexis.utils.warning("visible")
```

Output:

```
[WARNING] visible
```

Cork

C++: `phynexis::utils::wrappers::CorkWrapper` **Python:** `phynexis.utils.wrappers.CorkWrapper`

Header: `src/utils/wrappers/cork_wrapper.hpp`

Boolean operations on triangle meshes (intersection, union, difference, XOR) via the Cork library.

Description

`CorkWrapper` has no default constructor in Python. It is typically accessed through the `wrappers` submodule or used via the `Tessellation` class when `use_cork=True`.

Methods

`mesh_intersect(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute intersection of two meshes.

`mesh_union(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute union of two meshes.

`mesh_difference(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute difference ($\text{mesh1} - \text{mesh2}$).

`mesh_xor(v1, f1, v2, f2, out_v, out_f, out_info)`

Compute symmetric difference.

Parameters: | Parameter | Type | Description | |
| `v1, v2` | `list[Vec3d]` | Input mesh vertices | | `f1, f2` | `list[Vec3i]` | Input mesh faces | | `out_v` | `list[Vec3d]` | Output vertices (inout) | | `out_f` | `list[Vec3i]` | Output faces (inout) | | `out_info` | `list[int]` | Output labels (inout, optional) |

Note

- The overload without `out_info` is also available.
 - A specialized `mesh_intersect` with tolerance and direction exists for ray-like intersection tests.
 - `CorkWrapper` instance access in Python is currently limited. Consider using STLModel-based boolean operations if available.
-

Unexposed C++ API

- Default constructor for direct instantiation

DataIO

C++: `phynexis::utils::DataIO` **Python:** `phynexis.utils.DataIO` **Header:** `src/utils/serde/data_io.hpp`

Utility class for importing and exporting tabular data (CSV, TSV, space-separated) with automatic delimiter detection.

Methods

`import_data(filename, lines_to_skip=0)`

Import numeric data from a file. Supports comma, tab, and space delimiters with auto-detection.

Parameters: | Parameter | Type | Default | Description | |-----|-----|-----| | `filename` | `str` | — | Path to data file | | `lines_to_skip` | `int` | 0 | Number of header lines to skip |

Returns: `list[list[float]]` — 2D array of doubles

`save_data(data, head, filename)`

Save tabular data to a file with the given header string.

Parameters: | Parameter | Type | Description | |-----|-----|-----| | `data` | `list[list[int]]` or `list[list[float]]` | 2D data array | | `head` | `str` | Header line (e.g. "x,y,z") | | `filename` | `str` | Output file path |

`file_exist(filename)`

Check if a file exists.

Parameters: | Parameter | Type | Description | |-----|-----|-----| | `filename` | `str` | File path to check |

Returns: `bool`

Example

```
import phynexis

# Save integer data
phynexis.utils.DataIO.save_data(
    [[1, 2, 3], [4, 5, 6]],
    "x,y,z",
    "/tmp/data_int.csv"
)

# Save float data
phynexis.utils.DataIO.save_data(
    [[1.5, 2.5, 3.5], [4.5, 5.5, 6.5]],
    "x,y,z",
    "/tmp/data_float.csv"
)

# Import data (auto-detects delimiter)
data = phynexis.utils.DataIO.import_data("/tmp/data_float.csv")
print("rows:", len(data))
print("first row:", data[0])
print("second row:", data[1])
```

```
# File existence check  
print("exists:", phynexis.utils.DataIO.file_exist("/tmp/data_float.csv"))  
print("missing:", phynexis.utils.DataIO.file_exist("/tmp/nonexistent.csv"))
```

Output:

```
rows: 2  
first row: [1.5, 2.5, 3.5]  
second row: [4.5, 5.5, 6.5]  
exists: True  
missing: False
```

Unexposed C++ API

- `detect_delimiter()` / `parse_line()` —internal helpers

FlatHashMap

C++: `phynexis::utils::FlatHashMap<Key, Value>` **Python:** `phynexis.utils.FlatHashMap`
(alias for `FlatHashMap64_32`) **Header:** `src/utils/types/flat_hash_map.hpp`

Flat hash map with open addressing. Optimized for cache-friendly iteration and small data performance. Exposed as template instantiations with Pythonic dict-like interface.

Available Instantiations

Python Name	Key Type	Value Type	Description
<code>FlatHashMap</code>	<code>int64</code>	<code>int32</code>	Default alias (64→32)
<code>FlatHashMap64_32</code>	<code>int64</code>	<code>int32</code>	64-bit key, 32-bit value
<code>FlatHashMap64_64</code>	<code>int64</code>	<code>int64</code>	64-bit key, 64-bit value
<code>FlatHashMap32_32</code>	<code>int32</code>	<code>int32</code>	32-bit key, 32-bit value
<code>FlatHashMap64_64i</code>	<code>int64</code>	<code>int64</code>	Alias for <code>FlatHashMap64_64</code>

Constructor

`FlatHashMap()`

Creates an empty hash map with default capacity.

`FlatHashMap(bucket_count)`

Creates a hash map with pre-allocated bucket count.

Parameters: | Parameter | Type | Description | |---|---|---| | `bucket_count` | `int` | Initial number of buckets |

Methods

`empty()`

Return True if the map contains no elements.

`size() / __len__()`

Return the number of key-value pairs.

`max_size()`

Return the maximum possible size.

`reserve(count)`

Pre-allocate space for at least `count` elements.

`clear()`

Remove all elements.

insert(key, value)

Insert a key-value pair. Returns (inserted, value) tuple.

erase(key)

Remove the element with the given key. Returns number of elements removed (0 or 1).

find(key)

Lookup a key. Returns (found, value) tuple.

contains(key) / key in map

Check if a key exists.

count(key)

Return 1 if key exists, 0 otherwise.

at(key)

Access value by key with bounds checking.

load_factor()

Return current load factor (size / capacity).

max_load_factor() / max_load_factor(ml)

Get or set the maximum load factor threshold for rehashing.

rehash(count)

Force rehash to at least count buckets.

Python Protocols

map[key] (getitem)

Access or default-insert a value.

map[key] = value (setitem)

Insert or update a key-value pair.

key in map (contains)

Check membership.

for k, v in map (iteration)

Iterate over key-value pairs.

```
len(map)
```

Return element count.

Example

```
import phynexis

# Create default map (int64 -> int32)
m = phynexis.utils.FlatHashMap()
print("empty:", m.empty())

# Insert via setitem
m[10] = 100
m[20] = 200
m[30] = 300

print("size:", m.size())
print("m[10]:", m[10])
print("m.at(20):", m.at(20))
print("contains 10:", m.contains(10))
print("contains 99:", 99 in m)
print("count 10:", m.count(10))
print("len:", len(m))

# find
found, val = m.find(10)
print("find(10):", found, val)
found2, val2 = m.find(99)
print("find(99):", found2, val2)

# insert method
result = m.insert(40, 400)
print("insert(40, 400):", result)

# items / keys / values
print("items:", m.items())
print("keys:", m.keys())
print("values:", m.values())

# iteration
for k, v in m:
    print(f" {k} -> {v}")

# erase
m.erase(10)
print("after erase 10, size:", m.size())

# load factor
print("load_factor:", m.load_factor())

# reserve and rehash
m.reserve(100)
```



```

print("after reserve, size:", m.size())
m.rehash(256)
print("after rehash, load_factor:", m.load_factor())

# clear
m.clear()
print("after clear, size:", m.size())

# Bucket count constructor
m2 = phynexis.utils.FlatHashMap(64)
print("m2 size:", m2.size())

# 64->64 variant
m64 = phynexis.utils.FlatHashMap64_64()
m64[1] = 100
m64[2] = 200
print("64_64 size:", m64.size())

# 32->32 variant
m32 = phynexis.utils.FlatHashMap32_32()
m32[1] = 100
print("32_32 size:", m32.size())

```

Output:

```

empty: True
size: 3
m[10]: 100
m.at(20): 200
contains 10: True
contains 99: False
count 10: 1
len: 3
find(10): True 100
find(99): False 0
insert(40, 400): (True, 400)
items: [(20, 200), (10, 100), (30, 300), (40, 400)]
keys: [20, 10, 30, 40]
values: [200, 100, 300, 400]
  20 -> 200
  10 -> 100
  30 -> 300
  40 -> 400
after erase 10, size: 3
load_factor: 0.046875
after reserve, size: 3
after rehash, load_factor: 0.01171875
after clear, size: 0
m2 size: 0
64_64 size: 2
32_32 size: 1

```

Unexposed C++ API

The following methods are not yet exposed to Python:

- Copy/move constructors and assignment operators
 - `emplace()` / `try_emplace()` —template methods difficult to bind
 - Iterator-based `erase(pos)` —iterator not fully exposed
 - `cbegin()` / `cend()` —const iterators
-

Binding Fix Record

2026-05-03: `items()` / `keys()` / `values()` originally returned `VecX<T>` which threw `TypeError` because the type was not registered. Changed to return `py::list`. Issue fixed. See `discovery-log.md` for details.

LevelSetField

C++: `phynexis::utils::LevelSetField` **Python:** `phynexis.utils.LevelSetField` **Header:**
`src/utils/level_set_field.hpp`

Represents a level set function ϕ on a regular 3D grid. Used for implicit surface representation, interface tracking, and shape operations.

Constructor

`LevelSetField()`

Creates a level set field with default grid parameters: - Corner: $(-0.5, -0.5, -0.5)$ - Spacing: 0.05 - Dimensions: (21, 21, 21)

Methods

`get_corner()`

Return the grid corner position.

Returns: `Vec3d`

`get_spacing()`

Return the grid spacing.

Returns: `float`

`get_dimensions()`

Return the grid dimensions.

Returns: `Vec3i`

`set_corner(corner)`

Set the grid corner.

Parameters: | Parameter | Type | Description | |—|—|—| | `corner` | `Vec3d` | Lower corner of the grid |

`set_spacing(sp)`

Set the grid spacing.

Parameters: | Parameter | Type | Description | |—|—|—| | `sp` | `float` | Cell spacing |

`set_dimension(dim)`

Set the grid dimensions.

Parameters: | Parameter | Type | Description | |—|—|—| | `dim` | `Vec3u` | Number of cells in each dimension |

`get_phi(id) / set_phi(id, value) / phi(i, j, k)`

Access level set values at grid cells.

Parameters: | Parameter | Type | Description | | `id` | `Vec3u` | Grid cell index | | `i, j, k` | `int` | Individual indices | | `value` | `float` | New phi value |

`init_from_sdf_calculator(sdf)`

Initialize phi values by sampling an SDFCalculator.

`signed_distance(pos)`

Interpolate signed distance at an arbitrary point in space.

`gradient_interpolate(pos)`

Interpolate gradient at a point.

`gradient_minus(id) / gradient_plus(id)`

Compute gradient at a grid point using backward/forward differences.

`reinitialization() / reinitialization(iterations, dt)`

Reinitialize the level set function to maintain signed distance property.

`is_initialized()`

Check if phi_table has been allocated.

Returns:bool

Example

```
import phynexis

# Create and configure grid
lsf = phynexis.utils.LevelSetField()
lsf.set_corner(phynexis.utils.Vec3d(0.0, 0.0, 0.0))
lsf.set_spacing(0.1)
lsf.set_dimension(phynexis.utils.Vec3u(11, 11, 11))

print("corner:", lsf.get_corner())
print("spacing:", lsf.get_spacing())
print("dimensions:", lsf.get_dimensions())

# Set phi values
lsf.set_phi(phynexis.utils.Vec3u(5, 5, 5), 1.0)
print("phi at center:", lsf.phi(5, 5, 5))

# Query signed distance
pos = phynexis.utils.Vec3d(0.5, 0.5, 0.5)
print("sd at", pos, ":", lsf.signed_distance(pos))
```

Output:

```
corner: Vec3d(0, 0, 0)
spacing: 0.1
dimensions: Vec3i(11, 11, 11)
phi at center: 1.0
sd at Vec3d(0.5, 0.5, 0.5) : -0.5
```

Unexposed C++ API

- `get_phi_array()` —FlatArray3D wrapper
- `get_phi_table()` —Direct vector access
- `index_3d(i, j, k)` —Flat index computation

math

C++: `phynexis::utils::math` **Python:** `phynexis.utils.math` **Header:** `src/utils/math/math.hpp`

Mathematical constants, vector operations, and quaternion utilities.

Constants

Name	Value	Description
PI	3.14159...	Circle constant
EPSILON	2.22e-16	Machine epsilon for doubles
TOL	1e-6	Default numerical tolerance
HUGE_VALUE	1e20	Large sentinel value
SQRT_2	1.41421...	Square root of 2
SQRT_3	1.73205...	Square root of 3

Vector Operations

`dot(a, b)`

Compute dot product of two vectors.

Parameters: | Parameter | Type | Description | |-----|-----| | a | Vec3d | First vector | | b | Vec3d | Second vector |

Returns: float

`cross(a, b)`

Compute cross product of two vectors.

Returns: Vec3d

`norm_l2(v)`

Compute L2 norm (magnitude) of a vector.

Returns: float

`normalize(v)`

Normalize a vector in-place.

Quaternion

Module `phynexis.utils.math.quaternion` provides quaternion operations.

`from_rodrigues(angle, axis)`

Create a quaternion from angle-axis (Rodrigues) representation.

Parameters: | Parameter | Type | Description | |———|——|———| | `angle` | `float` | Rotation angle in radians | | `axis` | `Vec3d` | Rotation axis |

Returns: `Vec4d`

`to_rodrigues(q) / to_matrix(q) / from_matrix(m)`

Convert quaternion to/from Rodrigues, rotation matrix.

`multiply(a, b) / conjugate(q) / normalize(q) / add(a, b)`

Quaternion arithmetic.

Example

```
import phynexis

# Constants
print("PI:", phynexis.utils.math.PI)
print("EPSILON:", phynexis.utils.math.EPSILON)

# Vector ops
a = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
b = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
print("dot:", phynexis.utils.math.dot(a, b))
print("cross:", phynexis.utils.math.cross(a, b))
print("norm:", phynexis.utils.math.norm_l2(a))

# Quaternion
q = phynexis.utils.math.quaternion.from_rodrigues(
    phynexis.utils.math.PI / 2,
    phynexis.utils.Vec3d(0.0, 0.0, 1.0)
)
print("quaternion:", q)
```

Output:

```
PI: 3.141592653589793
EPSILON: 2.220446049250313e-16
dot: 0.0
cross: Vec3d(0, 0, 1)
norm: 1.0
quaternion: Vec4d(0, 0, 0.707107, 0.707107)
```

Unexposed C++ API

- `legendre_p`, `calculate_ynm_fast`, `normalization_factor` —spherical harmonics functions
- `flat_array3d` utilities

MiniMap

C++: `phynexis::utils::MiniMap<Int64, Int64>` **Python:** `phynexis.utils.MiniMapIdx64`

Header: `src/utils/mini_map.hpp`

Lightweight hash map optimized for index-to-index lookups. Uses flat hash map internally for cache-friendly access.

Constructor

`MiniMapIdx64()` Creates an empty map.

Methods

`exist(key)`

Check if a key exists in the map.

Parameters: | Parameter | Type | Description | |---|---|---| | `key` | `int` | Key to look up |

Returns: `bool` —True if key exists

`size()`

Return the number of entries.

Returns: `int` —Entry count

`clear()`

Remove all entries.

Python Protocols

`map[key] (getitem)`

Retrieve value by key. Raises `KeyError` if key not found.

`map[key] = value (setitem)`

Insert or update a key-value pair.

`key in map (contains)`

Check membership. Equivalent to `exist(key)`.

Example

```
import phynexis
```

```
m = phynexis.utils.MiniMapIdx64()
```

```
# Insert entries
```



```

m[10] = 100
m[20] = 200
m[30] = 300

print(f"size: {m.size()}")
print(f"m[10] = {m[10]}")
print(f"20 in map: {20 in m}")
print(f"99 in map: {99 in m}")
print(f"exist(10): {m.exist(10)}")

# Update
m[10] = 999
print(f"updated m[10] = {m[10]}")

# Clear
m.clear()
print(f"after clear: {m.size()}")

```

Output:

```

size: 3
m[10] = 100
20 in map: True
99 in map: False
exist(10): True
updated m[10] = 999
after clear: 0

```

Unexposed C++ API

MiniMap is a template class. Only the Int64→Int64 instantiation (MiniMapIdx64) is exposed. Other instantiations (e.g., int→double) are not available in Python.

OpenMP

C++: `phynexis::utils::parallel` / OpenMP runtime **Python:** `phynexis.utils` **Header:** `src/utils/parallel/parallel.hpp`

Thread-level parallelism control. These functions map directly to OpenMP runtime calls.

Functions

`omp_get_max_threads()`

Return the maximum number of threads available.

Returns: `int`

`omp_set_num_threads(num_threads)`

Set the number of threads for subsequent parallel regions.

Parameters: | Parameter | Type | Description | |---|---|---| | `num_threads` | `int` | Number of threads |

`parallel_max_threads()`

Alias for `omp_get_max_threads()`.

`parallel_set_num_threads(num_threads)`

Alias for `omp_set_num_threads()`.

Example

```
import phynexis

print("max threads:", phynexis.utils.omp_get_max_threads())
phynexis.utils.omp_set_num_threads(4)
print("set to 4 threads")
```

Output:

```
max threads: 10
set to 4 threads
```

Note

- These functions affect the **global** OpenMP thread pool.
- Setting threads via `parallel_set_num_threads()` also calls `omp_set_num_threads()` internally.

sampling

C++: `phynexis::utils::sampling` **Python:** `phynexis.utils.sampling` **Header:** `src/utils/sampling/sampling.hpp`

Sampling utilities for distributions, spatial points, orientations, and spherical arrangements.

Distribution Sampling

Class	Description
<code>UniformDistribution(min, max)</code>	Uniform distribution in [min, max]
<code>Normal(mean, std)</code>	Normal (Gaussian) distribution
<code>Exponential(lambda)</code>	Exponential distribution

Spatial Sampling

Spatial samplers generate points within a shape's bounding volume.

Class	Description
<code>Random(shape)</code>	Random uniform sampling inside a shape
<code>Grid(shape)</code>	Regular grid sampling
<code>CVT(shape)</code>	Centroidal Voronoi tessellation sampling
<code>WCVT(shape)</code>	Weighted CVT sampling
<code>Voronoi(shape)</code>	Voronoi-based sampling

Orientation Sampling

Class	Description
<code>Uniform</code>	Uniform random orientations
<code>ConeDir</code>	Orientations within a cone
<code>ConeRef</code>	Orientations relative to a reference direction

Spherical Sampling

Class	Description
<code>Uniform</code>	Uniform points on sphere
<code>CVT</code>	Spherical CVT
<code>WCVT</code>	Weighted spherical CVT
<code>GoldenSpiral</code>	Fibonacci/golden spiral point distribution

Example

```
import phynexis

# Distribution sampling
u = phynexis.utils.sampling.UniformDistribution(0.0, 1.0)
n = phynexis.utils.sampling.Normal(0.0, 1.0)
e = phynexis.utils.sampling.Exponential(1.0)
print("Uniform:", u)
print("Normal:", n)
print("Exponential:", e)
```

Output:

```
Uniform: <phynexis.lib.pyutils.sampling.distribution.Uniform object at 0x...>
Normal: <phynexis.lib.pyutils.sampling.distribution.Normal object at 0x...>
Exponential: <phynexis.lib.pyutils.sampling.distribution.Exponential object at 0x...>
```

Unexposed C++ API

- Distribution `sample()` methods returning scalar values
- Spatial samplers' `get_samples()` methods returning `VecX<Vec3d>` (unregistered)
- Orientation and spherical sampler detail methods
- Factory for creating samplers by name

SaveOptions / LoadOptions

C++: `phynexis::utils::SaveOptions, LoadOptions, SaveVtkOptions, FileFormat` **Python:**

`phynexis.utils.SaveOptions, LoadOptions, SaveVtkOptions, FileFormat` **Header:** `src/utils/serde/save_options`

Configuration objects for file save/load operations used by `Shape.save_to()` / `Shape.load_from()` and other serializable types.

FileFormat

Enum defining supported file formats.

Value	Description
<code>FileFormat.Auto</code>	Infer from file extension
<code>FileFormat.Json</code>	JSON format
<code>FileFormat.Binary</code>	Binary format
<code>FileFormat.VTK</code>	VTK format
<code>FileFormat.Stl</code>	STL mesh format
<code>FileFormat.Off</code>	OFF mesh format

SaveOptions

Constructor

`SaveOptions()` Default: `overwrite=True, create_directories=True, format_hint=Auto`.

Properties

Property	Type	Access	Description
<code>overwrite</code>	<code>bool</code>	read/write	Overwrite existing files
<code>create_directories</code>	<code>bool</code>	read/write	Create parent directories
<code>format_hint</code>	<code>FormatHint</code>	read/write	Format override hint

SaveVtkOptions

Inherits from `SaveOptions`. Adds VTK-specific settings.

Constructor

`SaveVtkOptions()` Default: `binary=False, use_double=False`.

Properties

Property	Type	Access	Description
<code>binary</code>	<code>bool</code>	read/write	Write VTK in binary mode
<code>use_double</code>	<code>bool</code>	read/write	Use double precision

LoadOptions

Constructor

LoadOptions() Default: create_directories=False, format_hint=Auto.

Properties

Property	Type	Access	Description
create_directories	bool	read/write	Create parent directories
format_hint	FormatHint	read/write	Format override hint

Example

```
import phynexis

# Save options with JSON format
opt = phynexis.utils.SaveOptions()
opt.overwrite = True
opt.create_directories = True
opt.format_hint.value = phynexis.utils.FileFormat.Json
print("format:", opt.format_hint.value)

# VTK options
vtk_opt = phynexis.utils.SaveVtkOptions()
vtk_opt.binary = True
vtk_opt.use_double = True
print("vtk binary:", vtk_opt.binary)

# Load options
load_opt = phynexis.utils.LoadOptions()
load_opt.create_directories = False
print("load create_dirs:", load_opt.create_directories)
```

Output:

```
format: FileFormat.Json
vtk binary: True
load create_dirs: False
```

SDFCalculator

C++: `phynexis::utils::SDFCalculator` **Python:** `phynexis.utils.SDFCalculator` **Header:**
`src/utils/sdf_calculator.hpp`

Computes signed distance fields (SDFs) from triangle meshes using an AABB tree accelerated by libigl. Supports point queries, surface projections, closest facet lookups, and batch evaluation.

Constructor

`SDFCalculator()`

Creates an empty calculator. Must be initialized via `init_from_stl()` before use.

Methods

`init_from_stl(vertices, facets)`

Initialize from raw vertex and face arrays.

Parameters: | Parameter | Type | Description | |—|—|—| | `vertices` | `list[Vec3d]` | Mesh vertex positions | | `facets` | `list[Vec3u]` | Triangle face indices |

`init_from_stl(stl_model)`

Initialize from an `STLModel` object.

Parameters: | Parameter | Type | Description | |—|—|—| | `stl_model` | `STLModel` | Loaded STL model |

`initialize()`

Reset to empty state.

`signed_distance(pos)`

Compute signed distance at a point. Negative = inside, positive = outside (depending on mesh orientation).

Parameters: | Parameter | Type | Description | |—|—|—| | `pos` | `Vec3d` | Query point |

Returns: `float` —Signed distance

`surface_projection(pos)`

Find the closest surface point on the mesh.

Parameters: | Parameter | Type | Description | |—|—|—| | `pos` | `Vec3d` | Query point |

Returns: `Vec3d` —Closest surface point

`closest_facet(pos)`

Find the index of the closest triangle facet.

Parameters: | Parameter | Type | Description | |—|—|—| | `pos` | `Vec3d` | Query point |

Returns: `int` —Facet index

`signed_distances(points)`

Batch compute signed distances for multiple points.

Parameters: | Parameter | Type | Description | |---|---|---| | `points` | `list[Vec3d]` | Query points |

Returns: `list[float]` —Signed distances

Example

```
import phynexis

# Load a mesh
model = phynexis.utils.STLModel()
model.init_from_stl("/path/to/sphere.stl")

# Build SDF
sdf = phynexis.utils.SDFCalculator()
sdf.init_from_stl(model)

# Query points
center = phynexis.utils.Vec3d(0.0, 0.0, 0.0)
outside = phynexis.utils.Vec3d(2.0, 0.0, 0.0)

print("sd at center:", sdf.signed_distance(center))
print("sd at (2,0,0):", sdf.signed_distance(outside))
print("surface proj:", sdf.surface_projection(outside))
print("closest facet:", sdf.closest_facet(outside))
```

Output:

```
sd at center: -0.49866546590344585
sd at (2,0,0): -2.2589289071338654
surface proj: Vec3d(-0.21319, 0.411084, -0.188573)
closest facet: 0
```

Unexposed C++ API

The following constructors and methods are not yet exposed:

- `SDFCalculator(const SDFCalculator&)` —Copy constructor
- `SDFCalculator(SDFCalculator&&)` —Move constructor
- Assignment operators

STLModel

C++: `phynexis::utils::STLModel` **Python:** `phynexis.utils.STLModel` **Header:** `src/utils/mesh/stl_model.hpp`

3D triangle mesh model supporting STL/OFF file I/O, geometric transformations, decimation, convex hull, and mesh analysis.

Constructor

`STLModel()`

Creates an empty model.

`STLModel(vertices, facets)`

Creates a model from vertex and face arrays.

Parameters: | Parameter | Type | Description | |---|---|---| | `vertices` | `list[Vec3d]` | Vertex positions | | `facets` | `list[Vec3i]` | Triangle face indices |

`STLModel(file_path)`

Load from a file path (auto-detects format).

Properties

Property	Type	Description
<code>vertices</code>	<code>list[Vec3d]</code>	Model vertices (read/write)
<code>facets</code>	<code>list[Vec3i]</code>	Triangle face indices (read/write)

Methods

`init_from_stl(file_path) / init_from_off(file_path)`

Load from STL or OFF file.

`save_to(path, file, options)`

Save model to file with format options.

`transpose() / rotate(rotation)`

Apply geometric transformations.

`copy_pose() / copy_pose_dev()`

Copy the model's current pose.

`remove_unreferenced_vertices() / remove_duplicate_vertices()`

Clean up mesh topology.

reorient_facets()

Ensure facet normals point outward.

decimate(target_num_faces)

Reduce face count via mesh decimation.

standardize()

Center at origin and scale to unit size.

set_size(size)

Scale to a specific size.

make_convex()

Convert to convex hull.

refine() / smooth_mesh(iterations)

Subdivide or smooth the mesh.

merge(other_model)

Merge with another STLModel.

size()

Return characteristic size (diagonal of bounding box).

get_center() / get_surface_area() / volume() / inertia()

Compute geometric properties.

is_convex() / is_face_outside(facet_index, point) / enclose(point)

Convexity and containment tests.

bound_aabb()

Return axis-aligned bounding box.

print_info()

Print model summary.

Static Methods

CenterOf(vertices, facets) / SurfaceAreaOf(vertices, facets) / VolumeOf(vertices, facets) / InertiaOf(vertices, facets) / check_convexity_of(vertices, facets)

Compute properties from raw mesh data without creating an STLModel instance.

Example

```
import phynexis

# Load from file
model = phynexis.utils.STLModel()
model.init_from_stl("/path/to/sphere.stl")

print("size:", model.size())
model.print_info()

# Geometric analysis
print("center:", model.get_center())
print("volume:", model.volume())
print("surface area:", model.get_surface_area())
print("is convex:", model.is_convex())

# Transformations
model.standardize()
print("after standardize, size:", model.size())
```

Output:

```
size: 0.08174285936076031
is convex: True
after standardize, size: 1.0
```

Example: Create from vertices and facets

```
import phynexis

model = phynexis.utils.STLModel()
model.vertices = [
    phynexis.utils.Vec3d(0.0, 0.0, 0.0),
    phynexis.utils.Vec3d(1.0, 0.0, 0.0),
    phynexis.utils.Vec3d(0.0, 1.0, 0.0),
]
model.facets = [phynexis.utils.Vec3i(0, 1, 2)]

print("vertices:", len(model.vertices))
print("facets:", len(model.facets))
print("center:", model.get_center())
print("volume:", model.volume())
print("surface area:", model.get_surface_area())
```

Output:

```
vertices: 3
facets: 1
center: Vec3d(0.333333, 0.333333, 0)
volume: 0.0
surface area: 0.5
```

Unexposed C++ API

- `get_triangle_strips()` —return type not fully exposed

StringUtils

C++: phynexis::utils::to_string **Python:** phynexis.utils.to_string **Header:** src/utils/string_utils.hpp
Numeric-to-string conversion with consistent formatting.

Functions

`to_string(value)`

Convert a numeric value to a formatted string.

Overloads: | Signature | Description | |---|---| | `to_string(value: int) -> str` | Integer formatting | |
`to_string(value: float) -> str` | Scientific notation for doubles |

Parameters: | Parameter | Type | Description | |---|---|---| | `value` | `int` | `float` | Numeric value to convert |

Returns: `str` —Formatted string

Example:

```
import phynexis

print(phynexis.utils.to_string(42))
print(phynexis.utils.to_string(3.14))
print(phynexis.utils.to_string(1e-6))
print(phynexis.utils.to_string(-2.5e3))
```

Output:

```
42
3.140000e+00
1.000000e-06
-2.500000e+03
```

Unexposed C++ API

The following C++ string utilities are not yet exposed to Python:

- `SplitString / JoinString` —string split/join
- `Trim / ToLower / ToUpper` —string transforms
- `FormatString` —formatted string (C++ `fmt` style)

SurfaceMesh

C++: `phynexis::utils::mesh::SurfaceMesh` **Python:** `phynexis.utils.SurfaceMesh` **Header:**
`src/utils/mesh/surface_mesh.hpp`

Minimal struct representing a surface mesh as vertices and triangular facets.

Constructor

`SurfaceMesh()`

Creates an empty mesh.

Properties

Property	Type	Description
<code>vertices</code>	<code>list[Vec3d]</code>	Vertex positions (read/write)
<code>facets</code>	<code>list[Vec3u]</code>	Triangle face indices (read/write)

Example

```
import phynexis

sm = phynexis.utils.SurfaceMesh()
sm.vertices = [
    phynexis.utils.Vec3d(0.0, 0.0, 0.0),
    phynexis.utils.Vec3d(1.0, 0.0, 0.0),
    phynexis.utils.Vec3d(0.0, 1.0, 0.0),
]
sm.facets = [phynexis.utils.Vec3u(0, 1, 2)]

print("vertices:", len(sm.vertices))
print("facets:", len(sm.facets))
print("first vertex:", sm.vertices[0])
```

Output:

```
vertices: 3
facets: 1
first vertex: Vec3d(0, 0, 0)
```

Unexposed C++ API

- `SurfaceMesh` is a simple POD struct. No additional methods are defined.

TimeUtils

C++: `phynexis::utils::get_time_micros` **Python:** `phynexis.utils.get_time_micros`

Header: `src/utils/time_utils.hpp`

High-resolution time measurement utilities.

Functions

`get_time_micros()`

Returns the current time in microseconds since an arbitrary epoch. Useful for measuring elapsed time between two calls.

Returns: `int` —Time in microseconds

Example:

```
import phynexis
import time

t0 = phynexis.utils.get_time_micros()
time.sleep(0.1) # 100 ms
t1 = phynexis.utils.get_time_micros()

elapsed_ms = (t1 - t0) / 1000.0
print(f"elapsed: {elapsed_ms:.2f} ms")
```

Output:

elapsed: 100.12 ms

Unexposed C++ API

- `Timer` class —Scoped RAII timer with automatic logging
- `SleepMicros` / `SleepMillis` —Sleep functions

Vectors and Matrices

C++: `phynexis::utils::Vec2, Vec3, Vec4, Vec6, Mat2, Mat3` **Python:** `phynexis.utils.Vec2d, Vec3d, Vec3i, Vec3u, Vec4d, Vec6d, Mat2d, Mat3d` **Header:** `src/utils/types/vec*.hpp, mat*.hpp`

Small fixed-size vector and matrix types used throughout phynexis. Exposed with Pythonic protocols (`len()`, indexing, iteration) where supported.

Vector Types

Python Name	C++ Type	Elements	Component Access	Arithmetic
<code>Vec2d</code>	<code>Vec2<double></code>	2 floats	<code>x, y</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec2i</code>	<code>Vec2<int></code>	2 ints	<code>x, y</code>	<code>+, -</code>
<code>Vec3d</code>	<code>Vec3<double></code>	3 floats	<code>x, y, z</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec3i</code>	<code>Vec3<int></code>	3 ints	<code>x, y, z</code>	<code>+, -</code>
<code>Vec3u</code>	<code>Vec3<Int64></code>	3 ints	<code>x, y, z</code>	<code>+, -</code>
<code>Vec4d</code>	<code>Vec4<double></code>	4 floats	<code>x, y, z, w</code>	<code>+, -, *, /</code> with scalar and vector
<code>Vec4i</code>	<code>Vec4<int></code>	4 ints	<code>x, y, z, w</code>	<code>+, -</code>
<code>Vec6d</code>	<code>Vec6<double></code>	6 floats	<code>xx, yy, zz, xy, xz, yz</code>	—

Constructors

`Vec2d(x, y) / Vec2i(x, y)`

`Vec3d(x, y, z) / Vec3i(x, y, z) / Vec3u(x, y, z)`

`Vec4d(x, y, z, w) / Vec4i(x, y, z, w)`

Note: The C++ `Vec4` constructor order is `(w, x, y, z)`, but the Python binding accepts `(x, y, z, w)` in the repr output order. After construction, `v.x, v.y, v.z, v.w` correspond to the expected components.

`Vec6d(xx, yy, zz, xy, xz, yz)` Symmetric tensor storage (e.g. stress/strain components).

Properties / Indexing

Feature	<code>Vec2d/3d/4d/2i/3i/4i/3u</code>	<code>Vec6d</code>
<code>len(v)</code>	Yes	No
<code>v[i] / v[i] = val</code>	Yes	No
Named components	Yes	Yes
<code>for x in v</code>	Yes (iteration)	No

Methods

`dot(other)` —**Vec3d only** Compute dot product.

Parameters: | Parameter | Type | Description | |———|——|———| | `other` | `Vec3d` | Other vector |

Returns: `float`

`cross(other)` —**Vec3d only** Compute cross product.

Parameters: | Parameter | Type | Description | |———|———| | `other` | `Vec3d` | Other vector |

Returns: `Vec3d`

Matrix Types

Python Name	C++ Type	Shape	Row Access
<code>Mat2d</code>	<code>Mat2<double></code>	2x2	<code>mat[i]</code> returns <code>Vec2d</code>
<code>Mat3d</code>	<code>Mat3<double></code>	3x3	<code>mat[i]</code> returns <code>Vec3d</code>

Constructors

`Mat2d()` / `Mat3d()` Creates a zero-initialized matrix.

Indexing

Feature	<code>Mat2d</code>	<code>Mat3d</code>
<code>mat[i]</code>	Returns <code>Vec2d</code>	Returns <code>Vec3d</code>
<code>mat[i] = row_vec</code>	Yes	Yes

Example

```
import phynexis

# Vec3d — most commonly used
v = phynexis.utils.Vec3d(1.0, 2.0, 3.0)
print("v:", v)
print("len:", len(v))
print("x:", v.x, "y:", v.y, "z:", v.z)
print("v[0]:", v[0], "v[1]:", v[1], "v[2]:", v[2])

# Element access and mutation
v.x = 10.0
v[1] = 20.0
print("after mutation:", v)

# Arithmetic
a = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
b = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
print("a + b:", a + b)
print("a * 2.0:", a * 2.0)
print("2.0 * a:", 2.0 * a)

# Dot and cross
print("dot:", a.dot(b))
print("cross:", a.cross(b))
```

```

# Vec3i
vi = phynexis.utils.Vec3i(1, 2, 3)
print("Vec3i:", vi, "len:", len(vi))

# Vec6d — symmetric tensor
s = phynexis.utils.Vec6d(1.0, 2.0, 3.0, 0.5, 0.6, 0.7)
print("Vec6d xx:", s.xx, "xy:", s.xy)

# Mat3d
m = phynexis.utils.Mat3d()
print("Mat3d:", m)
m[0] = phynexis.utils.Vec3d(1.0, 0.0, 0.0)
m[1] = phynexis.utils.Vec3d(0.0, 1.0, 0.0)
m[2] = phynexis.utils.Vec3d(0.0, 0.0, 1.0)
print("identity:", m)

```

Output:

```

v: Vec3d(1, 2, 3)
len: 3
x: 1.0 y: 2.0 z: 3.0
v[0]: 1.0 v[1]: 2.0 v[2]: 3.0
after mutation: Vec3d(10, 20, 3)
a + b: Vec3d(1, 1, 0)
a * 2.0: Vec3d(2, 0, 0)
2.0 * a: Vec3d(2, 0, 0)
dot: 0.0
cross: Vec3d(0, 0, 1)
Vec3i: Vec3i(1, 2, 3) len: 3
Vec6d xx: 1.0 xy: 0.5
Mat3d: Mat3d([0, 0, 0], [0, 0, 0], [0, 0, 0])
identity: Mat3d([1, 0, 0], [0, 1, 0], [0, 0, 1])

```

Known Issues

- Vec3u reports its Python type name as Vec3i (the repr also shows Vec3i). Functionally it works as unsigned 64-bit integers, but this is a binding quirk.
 - Vec6d does **not** support len(), indexing, or iteration. Only named component access (xx, yy, etc.) is available.
 - Vec4d constructor: the C++ order is (w, x, y, z), but Python passes (x, y, z, w). The repr output Vec4d(x, y, z, w) matches the property names, so Vec4d(1, 2, 3, 4) gives x=2, y=3, z=4, w=1 which is confusing. Prefer constructing with explicit intent or avoid Vec4d unless the w-component semantics are well understood.
-

Unexposed C++ API

- VecX<T> (dynamic-size vector) —used internally, some instantiations not registered for Python
- Matrix arithmetic operators (+, −, *) on Mat2d/Mat3d
- data() pointer access
- operator+= / -= / *= / /= compound assignments

Voronoi

C++: `phynexis::utils::voronoi` **Python:** `phynexis.utils.voronoi` **Header:** `src/utils/voronoi/*.hpp`

Voronoi tessellation on surfaces and in volumes. The Python API exposes `Tessellation` (static computation methods) and `Diagram` (result container).

TessellationParams

Configuration for Voronoi computation.

Constructor

`TessellationParams()` Defaults: `max_iter=1000`, `tol=0.001`, `use_cork=True`.

Properties

Property	Type	Description
<code>max_iter</code>	<code>int</code>	Maximum iterations
<code>tol</code>	<code>float</code>	Convergence tolerance
<code>use_cork</code>	<code>bool</code>	Use Cork for boolean clipping

Tessellation

Static methods for computing Voronoi diagrams.

`compute(vt_seeds, stl_model, params)`

Compute Voronoi tessellation from seed points on a surface mesh.

Parameters: | Parameter | Type | Description | |---|---|---| | `vt_seeds` | `list[Vec3d]` | Seed points | | `stl_model` | `STLModel` | Surface mesh | | `params` | `TessellationParams` | Optional parameters |

Returns: `Diagram`

`compute_centroidal(num_seeds, stl_model, params)`

Compute centroidal Voronoi tessellation (CVT) with automatic seed placement.

Parameters: | Parameter | Type | Description | |---|---|---| | `num_seeds` | `int` | Number of seeds | | `stl_model` | `STLModel` | Surface mesh | | `params` | `TessellationParams` | Optional parameters |

Returns: `Diagram`

Diagram

Result container for tessellation (no public constructor).

Example

```
import phynexis

# Load mesh
model = phynexis.utils.STLModel()
model.init_from_stl("/path/to/sphere.stl")

# Compute CVT
params = phynexis.utils.voronoi.TessellationParams()
params.max_iter = 100
params.tol = 0.01

diag = phynexis.utils.voronoi.Tessellation.compute_centroidal(10, model, params)
print("diagram:", diag)
```

Output:

```
diagram: <phynexis.lib.pyutils.voronoi.Diagram object at 0x...>
```

Unexposed C++ API

- SphericalDiagram, SphericalTessellation —Partially bound
- Tessellation::compute overload with implicit surfaces

vtk

C++: `phynexis::utils::vtk` **Python:** `phynexis.utils.vtk` **Header:** `src/utils/vtk/*.hpp`

VTK file I/O utilities for reading and writing point data, surface meshes, tetrahedral meshes, and line sets. Supports legacy VTK and VTK-HDF formats.

Data Classes

Class	Description
VtkData	Container for VTK dataset with points, cells, point data, cell data
VtkTopology	Cell connectivity and type information
PointData	Scalar/vector data attached to mesh points
CellData	Scalar/vector data attached to mesh cells

Read Functions

Function	Description
<code>read_points(filename)</code>	Read point cloud from VTK
<code>read_surface(filename)</code>	Read surface mesh from VTK
<code>read_tetrahedra(filename)</code>	Read tetrahedral mesh from VTK
<code>read_lines(filename)</code>	Read line/polygon data from VTK

Write Functions

Function	Description
<code>write_points(filename, points, point_data=None)</code>	Write point cloud to VTK
<code>write_surface(filename, vertices, facets, point_data=None, cell_data=None)</code>	Write surface mesh to VTK
<code>write_tetrahedra(filename, vertices, tets, point_data=None, cell_data=None)</code>	Write tetrahedral mesh to VTK
<code>write_with_lines(filename, vertices, lines, point_data=None)</code>	Write line set to VTK

Format Detection

Function	Description
<code>is_vtkhdf_format(filename)</code>	Check if file is VTK-HDF format
<code>detect_vtkhdf_format(filename)</code>	Detect VTK-HDF format variant

Example

```
import phynexis

# Format detection
print("is_vtkhdf:", phynexis.utils.vtk.is_vtkhdf_format("/path/to/data.vtkhdf"))

# Data classes exist but their constructors and methods may require
# additional VecX type registrations for full functionality.
```

Known Issues

- VtkData, PointData, CellData constructors and property access may fail if VecX<T> return types are not registered.
 - Some read/write functions expect VecX<Vec3d> or VecX<int> arguments which may not be registered.
-

Unexposed C++ API

- VtkReader / VtkWriter internal implementations
- VtkHDF reader/writer specialized for HDF5 backend
- format_detection internals

wrappers

C++: `phynexis::utils::wrappers` **Python:** `phynexis.utils.wrappers` **Header:** `src/utils/wrappers/*.hpp`

Thin wrappers around external libraries (Cork, CGAL, Eigen, igl, TetGen) providing mesh boolean operations, linear algebra, mesh processing, and tetrahedralization.

CorkWrapper

Boolean mesh operations via the Cork library.

Method	Description
<code>mesh_union(m1, m2)</code>	Union of two meshes
<code>mesh_intersect(m1, m2)</code>	Intersection of two meshes
<code>mesh_difference(m1, m2)</code>	Difference ($m1 - m2$)
<code>mesh_xor(m1, m2)</code>	Symmetric difference

Note: CorkWrapper has no default constructor exposed. Access methods via the class itself (static/class methods).

EigenWrapper

Linear algebra utilities via Eigen.

Method	Description
<code>solve_linear_eqn(A, b)</code>	Solve $Ax = b$
<code>get_eigen_vector(A, index)</code>	Get eigenvector by index

IGLWrapper

Mesh processing utilities via libigl.

Method	Description
<code>convex_hull(vertices, facets)</code>	Compute convex hull
<code>mesh_decimate(vertices, facets, target)</code>	Mesh simplification
<code>mesh_refine(vertices, facets)</code>	Mesh subdivision
<code>reorient_facets(vertices, facets)</code>	Ensure consistent facet orientation
<code>remove_duplicate_vertices(vertices, facets)</code>	Remove duplicate vertices
<code>remove_unreferenced_vertices(vertices, facets)</code>	Clean unreferenced vertices
<code>marching_cubes(field, corner, spacing, isovalue)</code>	Extract isosurface
<code>mesh_intersect(ray_origin, ray_dir, vertices, facets)</code>	Ray-mesh intersection
<code>facet_components(vertices, facets)</code>	Connected component labeling

Method	Description
<code>check_winding(vertices, facets)</code>	Check winding order
<code>get_points_inside_mesh(points, vertices, facets)</code>	Point-in-mesh test
<code>tetmesh_boundary(vertices, tets)</code>	Extract boundary surface from tetrahedral mesh

CGALWrapper

Computational geometry via CGAL.

Method	Description
<code>get_alpha_shape(points, alpha)</code>	Compute alpha shape
<code>get_tetmesh(vertices, facets)</code>	Tetrahedral mesh generation
<code>smooth_mesh(vertices, facets, iterations)</code>	Mesh smoothing

mpi

MPI wrapper utilities (if MPI is available).

Example

```
import phynexis

# CorkWrapper — access via class methods
try:
    # These require two STLModel instances
    result = phynexis.utils.wrappers.CorkWrapper.mesh_union(m1, m2)
except Exception as e:
    print("CorkWrapper requires valid mesh inputs")

# List available methods
print("CorkWrapper:", [x for x in dir(phynexis.utils.wrappers.CorkWrapper) if not x.startswith("_")])
print("IGLWrapper:", [x for x in dir(phynexis.utils.wrappers.IGLWrapper) if not x.startswith("_")])
print("CGALWrapper:", [x for x in dir(phynexis.utils.wrappers.CGALWrapper) if not x.startswith("_")])
```

Output:

```
CorkWrapper: ['mesh_difference', 'mesh_intersect', 'mesh_union', 'mesh_xor']
IGLWrapper: ['check_winding', 'convex_hull', 'facet_components', 'get_points_inside_mesh', 'marching_cubes']
CGALWrapper: ['get_alpha_shape', 'get_tetmesh', 'smooth_mesh']
```

Known Issues

- CorkWrapper has no default constructor. Methods must be called as `CorkWrapper.mesh_union(...)` rather than creating an instance.

- Most wrapper methods use `VecX<Vec3d>&` (mutable reference) arguments in their lambda bindings. Even though `VecX<T>` `type_caster` is now registered, `pybind11` cannot bind a temporary converted object to a non-const reference. Methods like `IGLWrapper.convex_hull`, `CorkWrapper.mesh_union`, etc. require binding fixes to accept values or use output tuples instead of output reference parameters.
 - `TetGenOptions` is exposed but its constructor and properties may have limited binding coverage.
-

Unexposed C++ API

- `EigenAdapter` / `EigenWrapper` internal matrix conversion helpers
- `mpi` wrapper full MPI collective operations
- `tetgen_get_tetmesh` factory function